

Keysight E36312A DC Sweep Manual de Usuario e Instalacion

ROMERUU-dev

2026-05-22

Contents

1	Objetivo	2
2	Requisitos	2
2.1	Software	2
2.2	Hardware	2
3	Instalacion desde GitHub	2
4	Ejecucion	3
5	Conexion al instrumento	3
6	Permisos USBTMC en Linux	3
7	Pestana Connection	4
8	Pestana Manual	4
9	Pestana Simple DC Sweep	4
10	Pestana Advanced DC Sweep	4
10.1	Orden de barrido	5
10.2	Tipos de barrido	5
10.3	Fuentes fijas	5
10.4	Presets	5
11	Mediciones de transistores	5
11.1	NMOS Output Curves	6
11.2	NMOS Transfer Curve	6
11.3	BJT Output Curves	6
12	Archivos exportados	6
13	Seguridad	7

14 Solucion de problemas	7
14.1 No aparece ningun recurso VISA	7
14.2 La ventana no muestra barra superior	7
14.3 El barrido se detiene por compliance	7
14.4 No se genera plot.png	7
15 Desarrollo y pruebas	8
16 Actualizacion en otra PC	8

1 Objetivo

Esta aplicacion controla una fuente Keysight E36312A/E36300 de tres canales por PyVISA/SCPI. Permite control manual, barridos DC simples, barridos DC avanzados con fuentes anidadas, graficas en tiempo real, exportacion de datos y apagado seguro de salidas.

La version final de la interfaz esta orientada a instrumento real. El modo de simulacion queda reservado para pruebas internas y no aparece en la interfaz de usuario.

2 Requisitos

2.1 Software

- Python 3.10 o superior.
- Git.
- Backend VISA: NI-VISA, Keysight IO Libraries o pyvisa-py.
- Dependencias Python listadas en `requirements.txt`.

2.2 Hardware

- Fuente Keysight E36312A o modelo compatible E36300.
- Cable USB, LAN o GPIB segun la configuracion del laboratorio.
- Para mediciones de transistores: protoboard o fixture, cables cortos, resistencias de proteccion y un punto de GND comun.

3 Instalacion desde GitHub

En una PC nueva, clona el repositorio y crea un entorno virtual:

```
git clone https://github.com/ROMERUU-dev/keysight_E36312A_DCSweep.git
cd keysight_E36312A_DCSweep
python -m venv .venv
```

En Linux o macOS:

```
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

En Windows PowerShell:

```
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Si en Linux el comando `python` no existe antes de activar el entorno, usa `python3`:

```
python3 -m venv .venv
source .venv/bin/activate
```

4 Ejecucion

Modo recomendado:

```
python app.py
```

La app abre maximizada, conservando la barra superior del sistema para mover, cerrar, minimizar y maximizar. Atajos y opciones:

Accion	Resultado
F11	Entra o sale de pantalla completa real.
Esc	Sale de pantalla completa real.
python app.py -windowed	Abre en ventana normal sin maximizar.
python app.py -fullscreen	Abre directamente en pantalla completa real.
python app.py -debug	Activa logging mas detallado de SCPI y aplicacion.

Nota: en pantalla completa real, GNOME/Wayland oculta la barra superior de la ventana. Para mover o cerrar con botones del sistema, sal de pantalla completa con F11 o Esc.

5 Conexion al instrumento

1. Conecta la fuente Keysight por USB, LAN o GPIB.
2. Abre la app.
3. En la pestana **Connection**, presiona **Refresh**.
4. Selecciona el recurso VISA detectado.
5. Si no aparece, escribe el recurso manualmente, por ejemplo:

```
USB0::0x2A8D::0x1102::<serial>::INSTR
```

Despues presiona **Connect**. La app apaga las salidas al conectar y muestra la respuesta de *IDN?. El ultimo recurso usado se guarda en `config.json`, archivo local ignorado por Git.

6 Permisos USBTMC en Linux

Si Linux ve el dispositivo pero PyVISA no lista el recurso, revisa:

```
lsusb | grep -i keysight
ls -l /dev/usbtmc* /dev/bus/usb/*/* 2>/dev/null | grep -E 'usbtmc|2a8d' || true
```

Regla udev sugerida:

```
sudo tee /etc/udev/rules.d/99-keysight-e36312a.rules >/dev/null <<'EOF'
SUBSYSTEM=="usb", ATTR{idVendor}=="2a8d", ATTR{idProduct}=="1102", GROUP="plugdev", MODE="0660"
KERNEL=="usbtmc*", ATTRS{idVendor}=="2a8d", ATTRS{idProduct}=="1102", GROUP="plugdev", MODE="0660"
EOF
sudo udevadm control --reload-rules
sudo udevadm trigger
```

Desconecta y reconecta la fuente despues de aplicar la regla. Para una prueba temporal, ajusta el bus/dispositivo real y usa:

```
sudo chmod g+rw /dev/usbtmc0 /dev/bus/usb/001/007
```

7 Pestana Connection

- **Refresh**: vuelve a buscar recursos VISA.
- **Connect**: abre el recurso seleccionado, apaga salidas y consulta *IDN?.
- **Disconnect**: ejecuta apagado seguro y cierra comunicacion.
- **Emergency Stop**: detiene barridos activos, intenta poner salidas en 0 V y apaga todos los canales.

8 Pestana Manual

La pestana Manual controla CH1, CH2 y CH3 individualmente.

- **Voltage**: voltaje objetivo del canal.
- **Current limit**: limite de corriente.
- **Set V**: envia el voltaje.
- **Set I limit**: envia el limite de corriente.
- **ON/OFF**: prende o apaga la salida del canal.
- **Measure**: lee voltaje y corriente medidos.
- **All OFF**: apaga todos los canales.
- **Ramp to 0 V and OFF**: baja voltaje a 0 V y apaga salidas.

9 Pestana Simple DC Sweep

Esta pestana ejecuta un barrido de voltaje en un solo canal.

1. Conecta el instrumento.
2. Selecciona el canal.
3. Define **V start**, **V stop**, **V step**, **I limit**, **Settle time** y **Compliance tolerance**.
4. Presiona **Start Sweep**.
5. Observa la grafica en tiempo real.
6. Presiona **Export CSV** si quieres guardar el barrido simple.

El CSV de barrido simple se guarda por defecto en:

`runs/sweeps/dc_sweep_latest.csv`

10 Pestana Advanced DC Sweep

Advanced DC Sweep permite configurar barridos tipo `.dc` con hasta tres fuentes logicas. Cada fuente logica se asigna a un canal fisico.

10.1 Orden de barrido

- **1st Source:** barrido rapido/interior.
- **2nd Source:** barrido exterior, util para familias de curvas.
- **3rd Source:** barrido mas externo.

Ejemplo:

```
.dc VDS 0 5 1 VGS 0 2 1
```

La app ejecuta todos los valores de VDS con VGS=0, despues todos los valores de VDS con VGS=1, y asi sucesivamente.

10.2 Tipos de barrido

- **Linear:** inicio, fin e incremento lineal.
- **Decade:** puntos por decada.
- **Octave:** puntos por octava.
- **List:** lista manual de valores.

10.3 Fuentes fijas

La seccion **Fixed Sources** permite definir cada canal como:

- **Off:** canal apagado.
- **Fixed voltage:** canal a voltaje fijo durante todo el barrido.
- **Swept source:** canal controlado por una fuente barrida.

La app valida que un canal no sea fuente fija y barrida al mismo tiempo.

10.4 Presets

- **Single Channel I-V:** barrido I-V de un canal.
- **Diode I-V:** barrido bajo para diodos, con limite sugerido de 5 mA a 10 mA.
- **NMOS Output Curves:** VDS en CH1 y VGS en CH2.
- **NMOS Transfer Curve:** VDS fijo en CH1 y VGS barrido en CH2.
- **BJT Output Curves:** VCE en CH1 y VBASE en CH2.

11 Mediciones de transistores

La pestana **Transistor** es una pantalla de etapa futura. Hoy las mediciones reales de transistores se hacen desde **Advanced DC Sweep** con presets.

11.1 NMOS Output Curves

Configuracion:

1st Source: VDS -> CH1, Linear, Start 0 V, Stop 5 V, Increment 0.05 V
2nd Source: VGS -> CH2, Linear, Start 0 V, Stop 3.3 V, Increment 0.1 V
3rd Source: disabled
X axis: Source1 value
Y axis: CH1 I_{meas}
Group curves by: Source2

Conexion:

CH1+ -> Drain
CH1- -> GND comun
CH2+ -> Gate
CH2- -> GND comun
Source -> GND comun

Interpretacion:

- ID se aproxima con la corriente medida en CH1.
- IG se observa con la corriente medida en CH2.
- Cada valor de VGS produce una curva ID contra VDS.

11.2 NMOS Transfer Curve

Usa VDS fijo en CH1 y barre VGS en CH2. Grafica:

- X: Source1 value o CH2 V_{meas}.
- Y: CH1 I_{meas}.
- Group curves by: None.

11.3 BJT Output Curves

Usa CH1 como VCE y CH2 como VBASE. Debe existir una resistencia externa de base para limitar corriente. La corriente de colector se observa como corriente en CH1. No conectes la base directamente a una fuente sin resistencia de proteccion.

12 Archivos exportados

Los barridos avanzados se exportan automaticamente en:

runs/YYYY-MM-DD_HH-MM-SS_<sweep_name>/

Contenido:

Archivo	Contenido
---------	-----------

<code>data.csv</code>	Datos por punto: setpoints, mediciones, potencia y compliance.
<code>metadata.json</code>	IDN, recurso VISA, tiempos, directiva .dc, fuentes y limites.
<code>sweep_config.json</code>	Configuracion del barrido para repetirlo despues.
<code>plot.png</code>	Captura de la grafica.
<code>log.txt</code>	Eventos y condiciones de parada.

13 Seguridad

- La app apaga salidas al conectar.
- Ante errores VISA/SCPI, intenta apagar salidas.
- Al cerrar, pregunta si debe poner salidas en 0 V y apagar.
- **Emergency Stop** debe usarse si el DUT entra en una condicion no deseada.
- Usa resistencias limitadoras y limites de corriente conservadores.

Limites por defecto:

Canal	Voltaje	Corriente	Potencia
CH1	0 a 6 V	0 a 5 A	30 W max
CH2	0 a 25 V	0 a 1 A	25 W max
CH3	0 a 25 V	0 a 1 A	25 W max

14 Solucion de problemas

14.1 No aparece ningun recurso VISA

- Verifica cable y encendido del instrumento.
- En Linux, revisa permisos USBTMC y regla udev.
- Instala o revisa el backend VISA.
- Prueba escribir manualmente el recurso VISA.

14.2 La ventana no muestra barra superior

Eso ocurre solo en pantalla completa real. Presiona F11 o Esc para volver a una ventana normal/maximizada con barra del sistema.

14.3 El barrido se detiene por compliance

El DUT esta demandando corriente cercana al limite configurado. Reduce voltaje, aumenta proteccion externa o ajusta el limite solo si es seguro para el circuito.

14.4 No se genera plot.png

El CSV y metadata son los archivos principales. Si la grafica no se guarda, revisa permisos de escritura en `runs/`. La medicion no se invalida por un fallo de captura de grafica.

15 Desarrollo y pruebas

Para correr pruebas:

```
source .venv/bin/activate
python -m pytest
```

Las pruebas cubren generacion de rangos, validacion de seguridad, exportacion, motor de barrido avanzado, layout principal y simulador interno.

16 Actualizacion en otra PC

Para actualizar una instalacion existente:

```
cd keysight_E36312A_DCSweep
git pull
source .venv/bin/activate
python -m pip install -r requirements.txt
python app.py
```